

LeadGen

The next-generation lead generation system

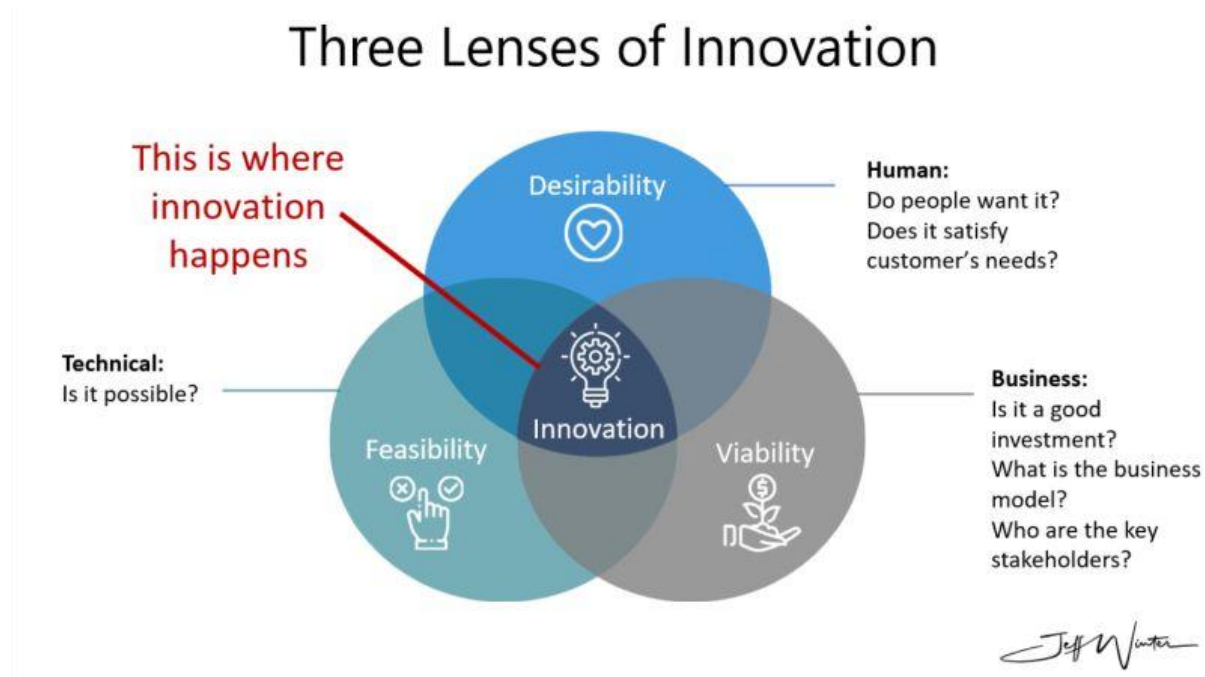
Contents

Goals.....	2
Problem statement.....	2
Philosophy.....	3
Agile Principles.....	3
Design Thinking Principles.....	4
Combined Philosophy.....	5
Empathise.....	5
User stories.....	6
Visitor.....	6
DevOps engineer.....	7
Support personnel.....	8
Site Reliability Engineer.....	9
QA Engineer.....	10
Front end Web Developer.....	12
Backend Developer.....	13
Data Engineer.....	14
Data Analyst.....	15
Define.....	16
Ideate.....	17
Prototype.....	17
Test.....	17
Implement.....	18
Overview.....	18
API.....	19
Solution Architecture.....	22
Sequence Diagram.....	24
Repositories.....	25
SDLC in context.....	25
SDLC.....	25
Relationship with Design Thinking.....	27
Overlapping Elements.....	27
Integration Points.....	28
Synergistic Approach.....	28
Integration in the Project.....	28

Goals

The goals of the work presented here are:

- Demonstrate competency in the core technical stack required from a backend engineer.
- Communicate maturity in the understanding of SDLC and product development end-to-end.
- Show how any work done (even for an interview), needs to be meaningful and be of value.
- Implement a project following Agile principles of fast iterative cycles and continuous improvement, embracing the principles of Design Thinking.
- Engage in an interesting project that is a good example of innovation, taking the three lenses of innovation into account.



Problem statement

Companies in various sectors have a need to generate leads from an advertisement, product showcase, or presentation of ideas for future products and services. It is desirable to collect information on potential customers, not only to gauge interest in the products or services presented but also to create a sales pipeline to maximise conversion rates and minimise customer acquisition costs.

Philosophy

The development philosophy that is followed here will make use of Agile and Design Thinking approaches. Combining Agile principles with a design thinking approach offers a holistic development philosophy that prioritises collaboration, iterative improvement, user-centricity, and flexibility in problem-solving. Below is a description of how Agile and Design Thinking align and guide the development process followed here.

Agile Principles



- Customer Collaboration over Contract Negotiation:
 - Design Thinking Alignment: Focuses on understanding user needs, involving users throughout the development process, and co-creating solutions.
- Responding to Change over Following a Plan:
 - Design Thinking Alignment: Embraces iteration, allowing flexibility to pivot based on user feedback and evolving requirements.
- Delivering Working Software Frequently:

- Design Thinking Alignment: Encourages rapid prototyping and iterative development to obtain quick feedback and refine solutions.
- Individuals and Interactions over Processes and Tools:
 - Design Thinking Alignment: Emphasises empathy and human-centred approaches, valuing people's insights over rigid processes.
- Working Solutions as a Measure of Progress:
 - Design Thinking Alignment: Encourages tangible results and prototypes that validate concepts and demonstrate progress.

Design Thinking Principles



- Empathy with Users:
 - Agile Alignment: Places importance on understanding users' needs, motivations, and pain points to drive solution development.
- Define Problem Statements:
 - Agile Alignment: Aligns with the Agile concept of a clear vision and scope, ensuring everyone understands the problem being solved.
- Ideate and Prototype:
 - Agile Alignment: Encourages iterative and collaborative ideation, rapid prototyping, and experimentation to explore various solutions.
- User-Centricity:
 - Agile Alignment: Aligns with Agile's focus on delivering value to end-users through continuous iterations and feedback.
- Test and Iterate:
 - Agile Alignment: Embraces the Agile concept of continuous improvement through testing, feedback, and iterative development cycles.

Combined Philosophy

- **User-Centric Approach:** Both Agile and design thinking prioritise understanding user needs and iterating based on user feedback.
- **Iterative Development:** Agile's iterations and design thinking's iterative ideation and prototyping complement each other for continuous improvement.
- **Collaborative Problem-Solving:** Both methodologies emphasise collaboration, encouraging cross-functional teams and diverse perspectives.
- **Flexibility and Adaptability:** Agile's responsiveness to change and design thinking's openness to iterating on ideas align for adaptable solutions.
- **Focus on Value Delivery:** Together, they aim to deliver value by solving real user problems through iterations and validated solutions.

When combined, Agile principles and design thinking create a powerful synergy, fostering a culture of innovation, empathy, and iterative improvement throughout the development process.

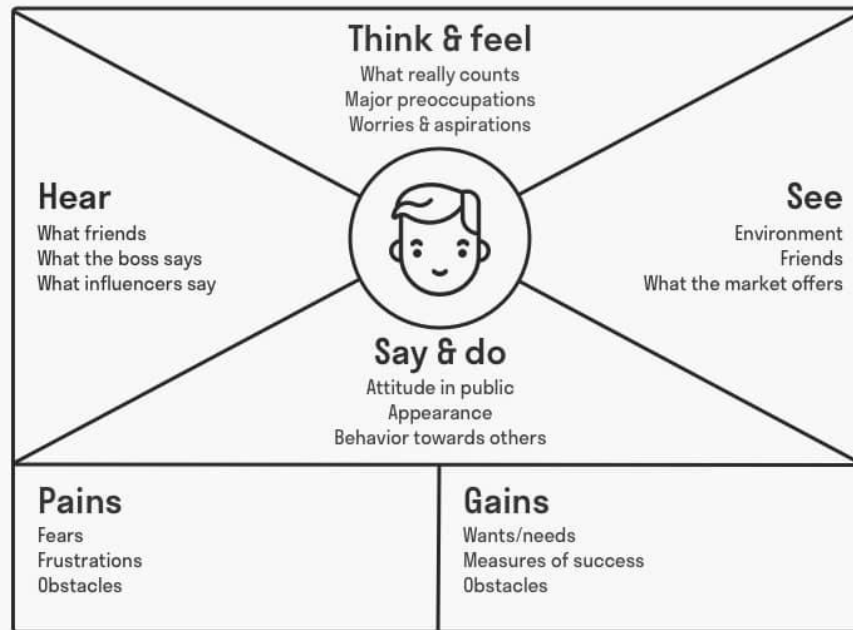
Empathise

In this stage the following activities take place:

- **Empathise with Users:** Understand the needs and pain points of users who will interact with the lead generation platform.
- **Define User Stories:** Create user stories or journey maps that represent how users will navigate through the lead submission process.
- **Non-functional requirements:** Develop a clear understanding of other factors that are important to stakeholders.

For the individual components in the system, the following needs to be done:

- Understand user needs and pain points in accessing and interacting with the lead submission form.
- Understand the need for a robust backend service that securely handles lead submissions and interacts with the database.
- Understand the need for a scalable, secure, and reliable infrastructure for hosting the application.
- Recognize the need for automated testing and deployment for efficient development cycles.



User stories

User stories are a way to capture functional requirements from the perspective of the end-users. Based on interactions with all stakeholders, the following user stories have been defined:

Visitor

As a Visitor, I Want To:

1. View Available Items
 - a. Scenario: As a user, I want to see a list of available items on the website.
 - b. Acceptance Criteria: Upon visiting the website, I should be able to view a list of items with their descriptions, and prices.
2. Select a Favourite Item
 - a. Scenario: As a user, I want to select my favourite item from the list displayed on the website.
 - b. Acceptance Criteria: I should be able to choose an item from the list by clicking on it or using a dropdown.
3. Submit Lead Information

- a. Scenario: After selecting an item, I want to provide my personal details to submit a lead.
 - b. Acceptance Criteria: I should be able to enter my name, surname, and email in a form and submit the information along with the selected item.
4. Receive Confirmation
 - a. Scenario: After submitting my details, I want confirmation that my lead has been successfully submitted.
 - b. Acceptance Criteria: Upon successful submission, I should receive a confirmation message indicating that my lead has been recorded.
5. Access Information Easily
 - a. Scenario: As a user, I want the website to be intuitive and easy to navigate.
 - b. Acceptance Criteria: The website should have clear navigation, guiding me to select an item and submit my information without confusion.
6. See Additional Information about Items
 - a. Scenario: As a user, I want more details about the items available.
 - b. Acceptance Criteria: I should be able to view additional information such as images, specifications, or reviews for each item by clicking or hovering over the item in the list.
7. Receive Error Handling
 - a. Scenario: If I make a mistake in submitting my details, I want to be notified and provided guidance on correcting the error.
 - b. Acceptance Criteria: If I miss filling out a required field or submit an incorrect email format, the website should prompt me to correct the mistake.

These user stories encapsulate various functionalities that a visitor would expect while interacting with the lead generation website, ensuring a smooth and intuitive user experience.

DevOps engineer

As a DevOps Engineer, I Want To:

1. Automate Deployment Processes
 - a. Scenario: As a DevOps engineer, I aim to automate deployment procedures to ensure smooth and efficient deployments.
 - b. Acceptance Criteria: Implement CI/CD pipelines using tools like GitHub Actions to automate the build, test, and deployment processes for both frontend and backend services.
2. Implement Infrastructure as Code (IaC)
 - a. Scenario: I want to manage infrastructure components using code to ensure consistency and scalability.
 - b. Acceptance Criteria: Utilise Terraform to define and deploy AWS infrastructure components (ECS, RDS, ALB, subnets) in a reproducible and automated manner.
3. Monitor System Performance

- a. Scenario: As a DevOps engineer, I need to monitor system health and performance to ensure optimal functionality.
 - b. Acceptance Criteria: Set up monitoring tools like CloudWatch to track system metrics, logs, and alarms for ECS, RDS, and other vital components.
4. Ensure High Availability and Fault Tolerance
 - a. Scenario: I aim to design a resilient architecture with redundancy and failover mechanisms for critical services.
 - b. Acceptance Criteria: Configure ECS with auto-scaling groups across multiple availability zones and RDS in multi-AZ mode to ensure high availability.
5. Implement Secure Access and Networking
 - a. Scenario: I want to secure the infrastructure and enable controlled access to services.
 - b. Acceptance Criteria: Configure private subnets, security groups, and network access controls, allowing access only through designated paths like ALB and NAT gateway.
6. Automate Parameter Management
 - a. Scenario: I want to manage sensitive information like database credentials securely and efficiently.
 - b. Acceptance Criteria: Store database credentials and other sensitive data in AWS Parameter Store and fetch them securely during deployment.
7. Enable Scalability and Flexibility
 - a. Scenario: I aim to design an infrastructure that can scale to accommodate increasing loads and adapt to changing demands.
 - b. Acceptance Criteria: Set up ECS with auto-scaling capabilities to dynamically adjust resources based on traffic and load fluctuations.

These user stories align with the DevOps engineer's objectives of automating processes, ensuring infrastructure reliability, security, scalability, and continuous monitoring for optimal system performance and reliability.

Support personnel

As Support Personnel, I Want To:

1. Access System Logs and Monitoring
 - a. Scenario: As support personnel, I need access to system logs and monitoring tools to troubleshoot issues efficiently.
 - b. Acceptance Criteria: Have access to CloudWatch or centralised logging systems to monitor system performance, diagnose errors, and identify potential issues.
2. Handle User Inquiries and Issues
 - a. Scenario: I want to address user inquiries or issues related to lead submissions promptly and effectively.
 - b. Acceptance Criteria: Have clear procedures and resources to respond to user inquiries via email, chat, or ticketing system and resolve issues within defined Service Level Agreements (SLAs).

3. Perform Routine Maintenance
 - a. Scenario: As part of maintenance tasks, I need to perform periodic checks and updates on the system.
 - b. Acceptance Criteria: Conduct routine checks for system health, perform backups, and apply updates or patches during maintenance windows to ensure system reliability.
4. Provide User Guidance and Support
 - a. Scenario: Users may require guidance on how to navigate the website or resolve basic issues.
 - b. Acceptance Criteria: Maintain a knowledge base, FAQs, or documentation to provide users with self-service options for common issues or guidance on using the lead submission platform.
5. Handle Incident Response
 - a. Scenario: In the event of system failures or critical incidents, I need to respond promptly and implement recovery measures.
 - b. Acceptance Criteria: Have defined incident response protocols, escalation procedures, and backup/restore plans to address critical incidents and minimise downtime.
6. Monitor User Feedback and Trends
 - a. Scenario: I want to gather and analyse user feedback to identify trends or recurring issues.
 - b. Acceptance Criteria: Collect and analyse user feedback, complaints, or trends to identify areas for improvement and provide input to the development team for enhancements.
7. Coordinate with Development Team
 - a. Scenario: For complex issues or feature requests, collaboration with the development team might be necessary.
 - b. Acceptance Criteria: Have channels for effective communication with the development team to escalate critical issues, report bugs, or request feature enhancements.

These user stories outline the support personnel's responsibilities in maintaining system health, providing user support, handling incidents, and ensuring continuous improvement by gathering and analysing user feedback.

Site Reliability Engineer

As a Site Reliability Engineer, I Want To:

1. Ensure High System Availability
 - a. Scenario: As an SRE, my primary goal is to ensure the system remains highly available and resilient.
 - b. Acceptance Criteria: Implement monitoring, alerting, and failover mechanisms to maintain high availability, minimising downtime and ensuring system uptime.
2. Optimise System Performance

- a. Scenario: I aim to optimise system performance for optimal user experience and efficient resource utilisation.
 - b. Acceptance Criteria: Conduct performance analysis, identify bottlenecks, and optimise system components (ECS, RDS, ALB) to enhance system performance and responsiveness.
3. Implement Disaster Recovery Strategies
 - a. Scenario: As an SRE, I need to design and implement disaster recovery and backup strategies.
 - b. Acceptance Criteria: Develop and test disaster recovery plans, ensuring data backups, and defining recovery procedures in case of catastrophic failures or data loss.
4. Conduct Chaos Engineering and Testing
 - a. Scenario: To proactively identify weaknesses and improve system resilience, I need to conduct chaos engineering and resilience testing.
 - b. Acceptance Criteria: Perform controlled experiments (such as chaos monkey tests) to simulate failures and ensure the system remains stable and resilient under adverse conditions.
5. Automate Routine Maintenance and Tasks
 - a. Scenario: Automating routine tasks and maintenance activities is essential for operational efficiency.
 - b. Acceptance Criteria: Automate tasks like system backups, security patches, and routine maintenance to reduce manual intervention and potential errors.
6. Continuously Improve Reliability
 - a. Scenario: As an SRE, I strive to improve system reliability through ongoing enhancements.
 - b. Acceptance Criteria: Analyze incident reports, perform root cause analysis, and propose system improvements to prevent similar incidents from occurring in the future.
7. Maintain Scalability and Elasticity
 - a. Scenario: I aim to ensure the system can seamlessly scale to handle increased loads.
 - b. Acceptance Criteria: Continuously monitor system performance metrics, implement auto-scaling strategies, and adjust resources dynamically to handle varying workloads.

These user stories align with an SRE's responsibilities, focusing on ensuring system reliability, performance optimization, disaster recovery planning, automation, and continuous improvement in system resilience and scalability.

QA Engineer

As a QA Engineer, I Want To:

1. Create Test Plans and Cases

- a. Scenario: I need to develop comprehensive test plans and test cases to validate system functionalities.
 - b. Acceptance Criteria: Create test plans covering frontend UI/UX, backend APIs, integration tests, and end-to-end scenarios, ensuring full coverage of functionalities.
2. Perform Functional Testing
 - a. Scenario: I aim to verify that all system functionalities work according to specifications.
 - b. Acceptance Criteria: Conduct functional testing to ensure that users can view cars, submit leads, receive confirmations, and navigate the website without issues.
3. Execute Regression Testing
 - a. Scenario: After each deployment or system update, I need to perform regression testing to ensure new changes haven't introduced defects.
 - b. Acceptance Criteria: Execute regression test suites to verify existing functionalities after updates, ensuring no regression or unintended changes.
4. Validate Error Handling and Edge Cases
 - a. Scenario: I want to ensure the system handles errors and edge cases gracefully without impacting user experience.
 - b. Acceptance Criteria: Test error scenarios like incorrect form submissions, network issues, or edge cases to ensure the system provides appropriate feedback and doesn't crash.
5. Test Performance and Load
 - a. Scenario: I aim to validate system performance under various loads to ensure it meets performance benchmarks.
 - b. Acceptance Criteria: Conduct performance and load testing using tools to validate system response times, throughput, and scalability under different traffic loads.
6. Verify Security and Compliance
 - a. Scenario: Ensuring system security and compliance with regulations is crucial.
 - b. Acceptance Criteria: Perform security testing, check for vulnerabilities, and validate compliance with security standards to safeguard user data and system integrity.
7. Provide Comprehensive Test Reports
 - a. Scenario: I need to document and report test results, findings, and bugs effectively.
 - b. Acceptance Criteria: Generate detailed test reports, document bugs with steps to reproduce, and work with the development team to ensure timely resolution.

These user stories outline the QA engineer's responsibilities in ensuring the lead generation system meets quality standards, performs reliably under different scenarios, and maintains security and compliance requirements.

Front end Web Developer

As a Front end Web Developer, I Want To:

1. Create User-Friendly UI Components
 - a. Scenario: Design and develop intuitive and user-friendly UI components for the lead generation website.
 - b. Acceptance Criteria: Develop reusable and responsive UI components using React, ensuring a pleasant user experience across devices and browsers.
2. Implement Car Listing Display
 - a. Scenario: Display the list of available cars with detailed information on the website.
 - b. Acceptance Criteria: Fetch and render car data dynamically, showcasing item details such as descriptions, prices, models, and years in a visually appealing manner.
3. Design Intuitive User Interaction
 - a. Scenario: Ensure seamless user interaction for selecting favourite items and submitting lead details.
 - b. Acceptance Criteria: Create intuitive forms and dropdowns allowing users to select items easily and submit lead information effortlessly, validating inputs for accuracy.
4. Integrate Backend API Calls
 - a. Scenario: Connect frontend components to backend APIs for data retrieval and lead submissions.
 - b. Acceptance Criteria: Implement API calls using TypeScript, enabling seamless communication between the frontend and backend services for data exchange.
5. Ensure Cross-Browser Compatibility
 - a. Scenario: Make sure the website functions consistently across various browsers and devices.
 - b. Acceptance Criteria: Test and ensure compatibility with major browsers (Chrome, Firefox, Safari, Edge) and devices (desktops, tablets, mobiles) for consistent behaviour.
6. Optimise Performance and Loading Speed
 - a. Scenario: Enhance website performance for faster loading and smoother user experience.
 - b. Acceptance Criteria: Optimise code, assets, and images, employing techniques like lazy loading and code splitting to improve website performance and reduce load times.
7. Collaborate with Design Team
 - a. Scenario: Work closely with designers to implement designs and ensure visual consistency.
 - b. Acceptance Criteria: Collaborate on design implementation, ensuring the website matches the provided design mockups and maintains a consistent look and feel.

These user stories outline the frontend developer's responsibilities in creating a visually appealing, intuitive, and responsive user interface, integrating frontend functionalities with backend services, and ensuring a seamless user experience across various platforms.

Backend Developer

As a Backend Developer, I Want To:

1. Implement RESTful APIs
 - a. Scenario: Develop backend APIs for handling lead submissions and retrieving item data.
 - b. Acceptance Criteria: Implement RESTful endpoints using Spring Boot and Java 11, allowing users to submit leads and retrieve item information from the database.
2. Design Database Schema
 - a. Scenario: Create a well-structured database schema to store lead and item information.
 - b. Acceptance Criteria: Design and implement database tables using PostgreSQL, defining relationships between leads and items for efficient data storage and retrieval.
3. Handle Lead Submissions
 - a. Scenario: Receive lead data from the front end and store it securely in the database.
 - b. Acceptance Criteria: Implement API endpoints to receive lead details, validate input, and persist the data securely in the leads table while maintaining data integrity.
4. Retrieve Car Information
 - a. Scenario: Fetch car details from the database and serve them to the front end.
 - b. Acceptance Criteria: Develop API endpoints to retrieve item information, ensuring accurate and updated data is provided to the front end based on user requests.
5. Implement Authentication and Authorization
 - a. Scenario: Secure backend APIs and endpoints to allow authorised access only.
 - b. Acceptance Criteria: Implement authentication mechanisms (e.g., JWT) and authorization checks to protect APIs, ensuring access is granted based on user roles and permissions.
6. Ensure API Reliability
 - a. Scenario: Make sure backend APIs are reliable, performant, and handle errors gracefully.
 - b. Acceptance Criteria: Implement error handling, logging, and validation to ensure robustness, scalability, and stability of the backend services.
7. Collaborate with Frontend Team
 - a. Scenario: Collaborate with frontend developers to integrate backend functionalities seamlessly.

- b. Acceptance Criteria: Work closely with frontend developers, providing necessary API documentation and support for smooth integration of frontend and backend components.

These user stories outline the backend developer's responsibilities in building secure, reliable, and efficient backend services, handling data storage, and retrieval, and ensuring seamless communication with the frontend components.

Data Engineer

As a Data Engineer, I Want To:

1. Design Database for Scalability
 - a. Scenario: Design a scalable and efficient database structure to handle growing data volumes.
 - b. Acceptance Criteria: Implement a well-optimised+ database schema using PostgreSQL with considerations for scalability, indexing, and performance tuning.
2. Implement Data Pipelines
 - a. Scenario: Create data pipelines to ingest, transform, and load lead and item data into the database.
 - b. Acceptance Criteria: Develop ETL (Extract, Transform, Load) processes using tools like Apache Kafka or Apache NiFi to handle data ingestion and transformation efficiently.
3. Ensure Data Quality and Integrity
 - a. Scenario: Maintain high data quality and integrity to ensure accurate reporting and analytics.
 - b. Acceptance Criteria: Implement data validation, cleansing, and quality checks to ensure consistency and accuracy of data stored in the database.
4. Enable Data Analytics and Reporting
 - a. Scenario: Facilitate data analytics and reporting by creating necessary data structures and tools.
 - b. Acceptance Criteria: Design data structures and provide tools for reporting and analytics teams to perform data analysis and generate insights from lead and item data.
5. Implement Data Security Measures
 - a. Scenario: Ensure data security and compliance with data protection regulations.
 - b. Acceptance Criteria: Implement encryption, access controls, and audit trails to safeguard sensitive data stored in the database, adhering to compliance requirements.
6. Collaborate with Development Teams
 - a. Scenario: Collaborate with backend and frontend teams to optimise data storage and retrieval processes.
 - b. Acceptance Criteria: Work with development teams to optimise queries, improve data access patterns, and ensure efficient utilisation of database resources.
7. Handle Data Migration and Versioning

- a. Scenario: Manage data migration and versioning for seamless updates and changes in the database structure.
- b. Acceptance Criteria: Develop strategies and tools to handle database schema changes, ensuring smooth migration and versioning without disrupting ongoing operations.

These user stories highlight the data engineer's responsibilities in designing, managing, and optimising the database infrastructure, ensuring data quality, security, and facilitating data-driven decision-making processes within the lead generation system.

Data Analyst

As a Data Analyst, I Want To:

1. Analyse Lead Conversion Rates
 - a. Scenario: Analyse lead data to understand conversion rates and trends.
 - b. Acceptance Criteria: Retrieve and analyse lead data from the database to calculate conversion rates, identify patterns, and provide insights into lead conversion performance.
2. Generate Reports and Dashboards
 - a. Scenario: Create visual reports and dashboards to communicate key metrics and insights.
 - b. Acceptance Criteria: Develop interactive dashboards using tools like Tableau or Power BI, displaying lead generation metrics, trends, and performance indicators.
3. Perform Cohort Analysis
 - a. Scenario: Conduct cohort analysis to understand user behaviour over time.
 - b. Acceptance Criteria: Segment leads based on acquisition time and analyse their behaviour, retention, and conversion patterns to identify trends and opportunities.
4. Identify User Preferences
 - a. Scenario: Analyse data to understand user preferences and favourite item choices.
 - b. Acceptance Criteria: Analyse lead data to identify popular item models, preferences, and correlations between user demographics and their choices.
5. Predictive Analysis for Leads
 - a. Scenario: Utilise machine learning or predictive models to forecast potential lead generation trends.
 - b. Acceptance Criteria: Develop predictive models using historical lead data to forecast future trends, aiding in strategic planning and resource allocation.
6. Optimise Marketing Strategies
 - a. Scenario: Analyse lead sources and channels to optimise marketing strategies.
 - b. Acceptance Criteria: Analyse data to identify effective marketing channels, campaigns, or sources generating high-quality leads, and guiding marketing strategy optimizations.
7. Collaborate with Development Teams

- a. Scenario: Collaborate with development teams to understand data requirements and optimise data access.
- b. Acceptance Criteria: Work with backend developers to define data schemas, and access patterns, and ensure data availability for analysis.

These user stories outline the data analyst's responsibilities in leveraging data to generate insights, optimise lead generation strategies, and provide valuable information for decision-making within the lead generation system.

Define

The activities in this stage include:

- Define the user interface elements required for easy data input and item selection.
- Define the functionalities required to process and validate lead submissions and interact with the database.
- Define the required AWS services, network configurations, and deployment strategies.
- Define the requirements for CI/CD, including testing procedures and deployment strategies.

Design a solution that can present items to a potential customer, capturing their personal details, to be persisted for analysis and use later.

"How Might We" (HMW) statements

HMW statements are used in design thinking to reframe challenges as opportunities for ideation and problem-solving. Here are some HMW statements related to the lead generation system:

- How might we improve the user experience for selecting favourite items on the website?
- How might we increase lead submission rates without compromising user data security?
- How might we optimise the backend APIs to ensure faster data retrieval for item details?
- How might we leverage user data to personalise item recommendations and enhance user engagement?
- How might we streamline the lead submission process to minimise user errors and maximise completion rates?
- How might we design intuitive dashboards for data analysts to derive actionable insights more effectively?
- How might we ensure continuous scalability of the database infrastructure as the user base and data volume grow?
- How might we proactively identify and address potential bottlenecks in the system before they impact user experience?
- How might we enhance data validation mechanisms to maintain high data quality and integrity?
- How might we improve cross-browser compatibility to ensure consistent performance across all major browsers and devices?

These HMW statements serve as prompts for brainstorming and ideation sessions to explore potential solutions and innovations for improving various aspects of the lead generation system.

Ideate

Collaborate with team members to generate ideas for enhancing user experience, improving form submission, and facilitating item selection.

- Brainstorm various UI/UX elements, layouts, and interactions to enhance user experience during form submission.
- Brainstorm various backend logic and data handling processes for seamless lead management.
- Brainstorm infrastructure design options and select technologies that meet scalability and security requirements.
- Brainstorm CI/CD workflows and strategies that fit project requirements.

Based on brainstorming sessions, the following idea is an attempt at the solution of the problem.

There will be a website for lead generation. This will be used to gauge interest in new products or services. The web page will be a single-page web application that can present a carousel of images, with a feedback form where a visitor fills in their name, surname, and email address, and chooses their favourite item, to be persisted for analysis and use later.

Prototype

Develop a proof-of-concept, something that users can see and feel, but it may not be fully working yet. It needs to convey the vision and provide an opportunity to provide feedback based on usage.

- Use Figma or wireframing tools to create low-fidelity prototypes of the web page, backend workflows, and infrastructure layout.
- Create a high-level architectural design for the backend service.
- Create infrastructure diagrams and Terraform configurations as prototypes.
- Create CI/CD pipeline configurations as prototypes.

Test

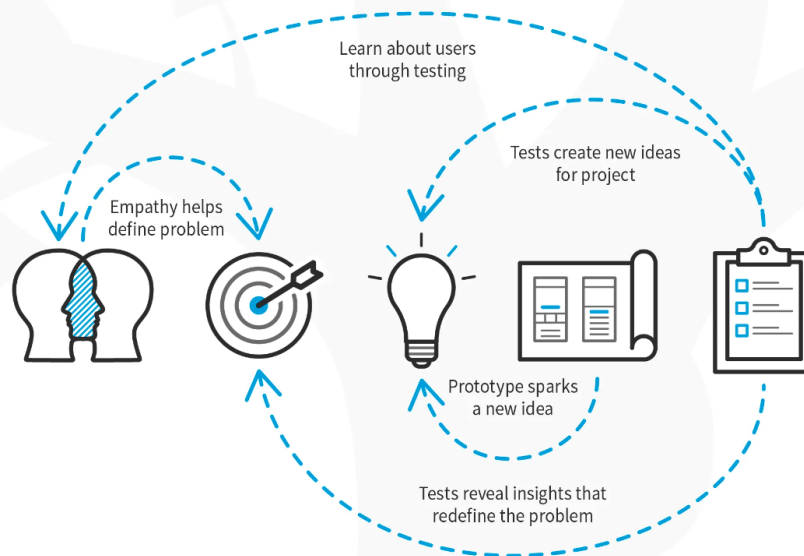
Share prototypes with potential users or stakeholders to gather feedback on usability, clarity, and efficiency. The idea is to get a consensus on the requirements and whether the solution fulfils the needs of the customer and other stakeholders before a significant development effort is incurred.

- Gather feedback from potential users on the usability and intuitiveness of the web page design.

- Conduct unit tests for functionalities, ensuring robustness and accuracy.
- Ensure the infrastructure setup meets performance, security, and scalability standards.
- Test the CI/CD pipeline with different scenarios and integration points.

Based on feedback, iterate on designs, functionalities, and infrastructure elements to enhance user experience and streamline processes. Even after the project's initial launch, collect user feedback and iterate on features or design elements for continuous improvement.

Design Thinking: A Non-Linear Process



Interaction Design Foundation
interaction-design.org

Implement

Overview

To get the list of items, the website will invoke a GET request on the /items endpoint, which returns the item ID, description, and price, read from an items table. When the user submits their choice, the lead is saved in the leads table through a POST request, with each row containing a timestamp, name, surname, email and item ID being the foreign key into the items table's ID column. An endpoint will allow all leads to be returned from the database through a

GET request. These REST APIs will be implemented in Spring Boot, which will communicate with the leads database. The APIs need to expose Swagger API documentation.

React and Typescript will be used for the web page. Spring Boot will be used for the backend with Java 11 and Postgres as the database.

Github Actions is to be used for the CI/CD pipeline with Github. The front end will be deployed to S3 and the Spring Boot application to a Docker container running on AWS ECS. Unit tests will be run for the Spring Boot service, using JUnit, on a pull request, before Github allows a feature branch or develop branch to be merged. When the develop branch is merged, a new Docker container is to be built on Github, pushed to ECR, and manually deployed on the workflow.

The ECS service must run in an auto-scaling group in 2 availability zones. The Postgres database must be in RDS multi-AZ. The Spring Boot service must log into CloudWatch.

The ECS service will be in a private subnet and communicate out to the Internet with a NAT gateway. RDS is also in the private subnet. The private subnet spans two availability zones. An application load balancer will route incoming requests to the Docker containers in the auto-scaling group. AWS certificate manager will manage the SSL certificate for the Elastic IP of the ALB. Route53 is used for the DNS of the load balancer. The ECS containers will read the database URL, username and password from the AWS Parameter Store.

To create the infrastructure, Terraform will be used for infrastructure as code.

API

1. Create repository on Github
<https://github.com/JanVDL1975/leadgen-api>
2. Generate base Spring Boot project

The image shows the Spring Initializr web interface. On the left, under 'Project', 'Maven' is selected. Under 'Language', 'Java' is selected. Under 'Spring Boot', '3.1.5' is selected. The 'Project Metadata' section shows: Group: com.leadgen, Artifact: api, Name: api, Description: LeadGen is the next-generation lead generation system. This is the repository, Package name: com.leadgen.api, Packaging: Jar, and Java version: 11. On the right, the 'Dependencies' section lists: Spring Web (WEB), Spring Data JPA (SQL), PostgreSQL Driver (SQL), Spring Boot DevTools (DEVELOPER TOOLS), and Lombok (DEVELOPER TOOLS). At the bottom, there are buttons for 'GENERATE', 'EXPLORE', and 'SHARE...'.

3. Create database (locally) and user:

```
$ psql -U postgres
postgres=# CREATE DATABASE leadgen WITH ENCODING 'UTF8';
postgres=# CREATE USER lguser WITH ENCRYPTED PASSWORD 'p@55w0rd';
postgres=# GRANT ALL PRIVILEGES ON DATABASE leadgen TO lguser;
postgres=# \c leadgen
leadgen=# \q
```

```
$ psql -W -U lguser leadgen
leadgen=> \q
```

4. Docker commands:

```
$ docker build -t leadgen-api .
$ docker tag leadgen-api jvanderlith085/leadgen-api:latest
$ docker push jvanderlith085/leadgen-api:latest
$ docker exec -it postgres:latest /bin/bash
```

5. Create database (on RDS) and user:

```
$ psql -U postgres -p 5432 -h leadgen.cbeqxs1473dx.af-south-1.rds.amazonaws.com
Password for user postgres:
psql (14.8 (Homebrew), server 16.1)
WARNING: psql major version 14, server major version 16.
Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256,
compression: off)
Type "help" for help.
```

```
postgres=> \l
```

```

                                List of databases
  Name      | Owner   | Encoding | Collate  | Ctype    | Access privileges
-----+-----+-----+-----+-----+-----
 postgres   | postgres | UTF8     | en_US.UTF-8 | en_US.UTF-8 |
 rdsadmin   | rdsadmin | UTF8     | en_US.UTF-8 | en_US.UTF-8 | rdsadmin=CTc/rdsadmin+
            |          |          |             |             | rdstopmgr=Tc/rdsadmin
 template0   | rdsadmin | UTF8     | en_US.UTF-8 | en_US.UTF-8 | =c/rdsadmin +
            |          |          |             |             | rdsadmin=CTc/rdsadmin
```

```

template1 | postgres | UTF8      | en_US.UTF-8 | en_US.UTF-8 | =c/postgres      +
          |           |           |             |             | postgres=CtC/postgres
(4 rows)

```

```

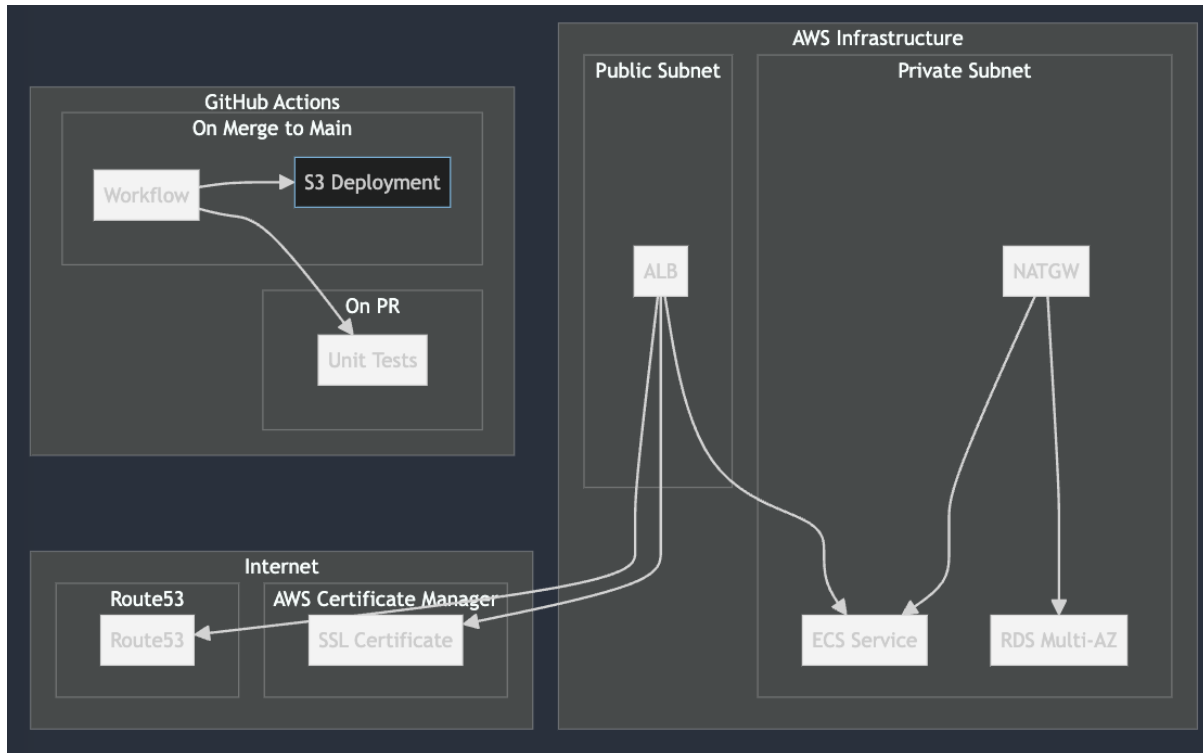
postgres=> CREATE DATABASE leadgen WITH ENCODING 'UTF8';
CREATE DATABASE
postgres=> CREATE USER lguser WITH ENCRYPTED PASSWORD 'p@55w0rd';
CREATE ROLE
postgres=> GRANT ALL PRIVILEGES ON DATABASE leadgen TO lguser;
GRANT
postgres=> \c leadgen
psql (14.8 (Homebrew), server 16.1)
WARNING: psql major version 14, server major version 16.
        Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256,
compression: off)
You are now connected to database "leadgen" as user "postgres".
leadgen=> \q

$ psql -W -U lguser leadgen -p 5432 -h
leadgen.cbeqxs1473dx.af-south-1.rds.amazonaws.com
Password:
psql (14.8 (Homebrew), server 16.1)
WARNING: psql major version 14, server major version 16.
        Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256,
compression: off)
Type "help" for help.

leadgen=> \q

```

Solution Architecture



<https://mermaid.live/>

flowchart TB

subgraph "AWS Infrastructure"

style inf style filled

style inf fill:#f9f9f9,stroke:#dddddd,stroke-width:1px

subgraph "Private Subnet"

style privSub style filled

style privSub fill:#ffffff,stroke:#dddddd,stroke-width:1px

NATGW-->ECS[ECS Service]

NATGW-->RDS[RDS Multi-AZ]

end

subgraph "Public Subnet"

style pubSub style filled

style pubSub fill:#ffffff,stroke:#dddddd,stroke-width:1px

ALB-->ECS

end

end

```

subgraph "Internet"
    style inet style filled
    style inet fill:#f9f9f9,stroke:#dddddd,stroke-width:1px

    subgraph "Route53"
        style route53 style filled
        style route53 fill:#ffffff,stroke:#dddddd,stroke-width:1px
        ALB-->DNS[Route53]
    end

    subgraph "AWS Certificate Manager"
        style certMan style filled
        style certMan fill:#ffffff,stroke:#dddddd,stroke-width:1px
        ALB-->SSL[SSL Certificate]
    end
end

subgraph "GitHub Actions"
    style ghActions style filled
    style ghActions fill:#f9f9f9,stroke:#dddddd,stroke-width:1px
    subgraph "On Merge to Main"
        style merge style filled
        style merge fill:#ffffff,stroke:#dddddd,stroke-width:1px
        Workflow-->S3Deploy[S3 Deployment]
    end

    subgraph "On PR"
        style pr style filled
        style pr fill:#ffffff,stroke:#dddddd,stroke-width:1px
        Workflow-->JUnit[Unit Tests]
    end
end

style ECS fill:#f3f3f3,stroke:#dddddd,stroke-width:1px
style ALB fill:#f3f3f3,stroke:#dddddd,stroke-width:1px
style RDS fill:#f3f3f3,stroke:#dddddd,stroke-width:1px
style DNS fill:#f3f3f3,stroke:#dddddd,stroke-width:1px
style SSL fill:#f3f3f3,stroke:#dddddd,stroke-width:1px
style Workflow fill:#f3f3f3,stroke:#dddddd,stroke-width:1px
style JUnit fill:#f3f3f3,stroke:#dddddd,stroke-width:1px

```

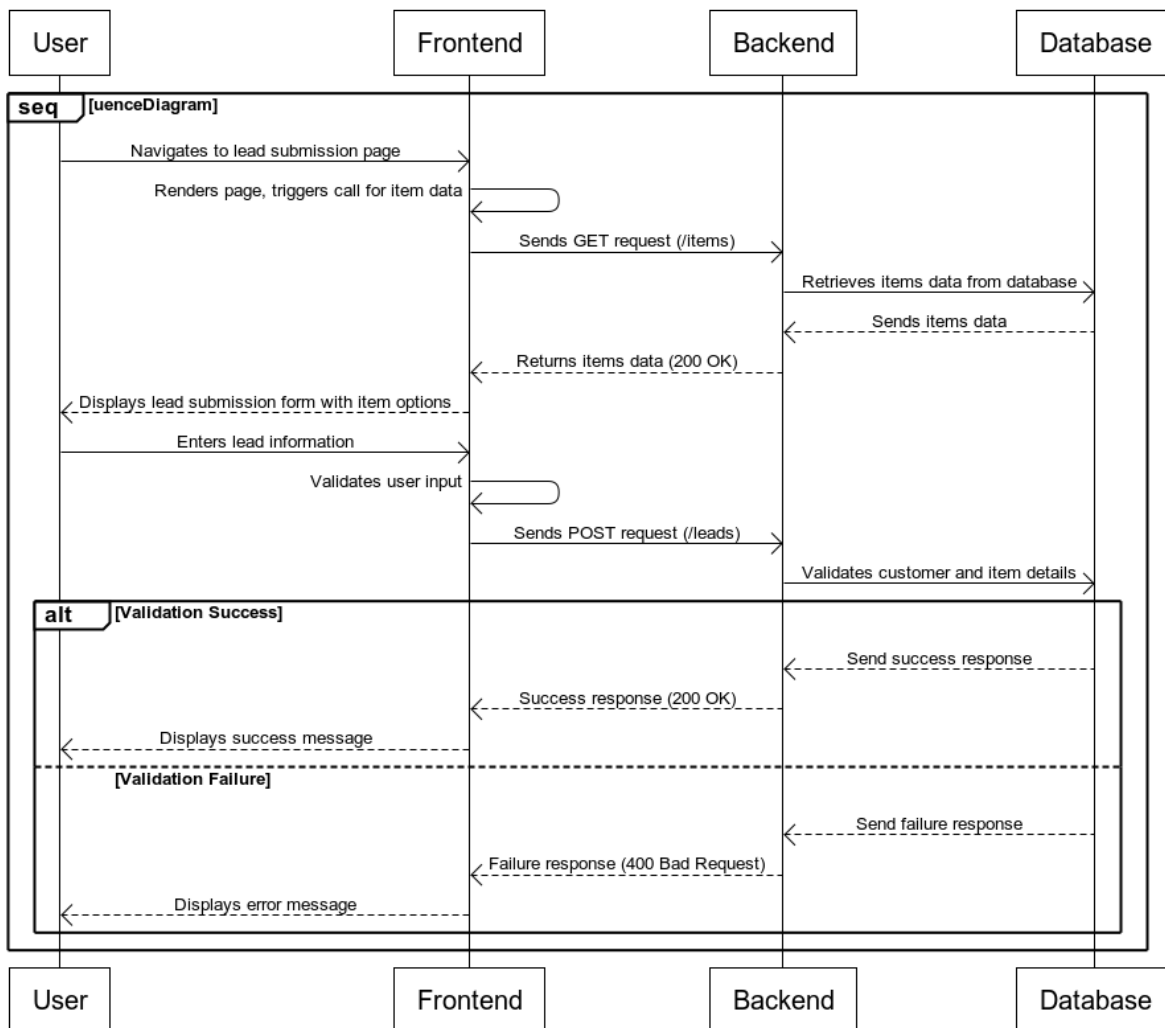
```

style privSub fill:#f3f3f3,stroke:#dddddd,stroke-width:1px
style pubSub fill:#f3f3f3,stroke:#dddddd,stroke-width:1px
style route53 fill:#f3f3f3,stroke:#dddddd,stroke-width:1px
style certMan fill:#f3f3f3,stroke:#dddddd,stroke-width:1px

style NATGW fill:#f3f3f3,stroke:#dddddd,stroke-width:1px

```

Sequence Diagram



www.websequencediagrams.com

```

sequenceDiagram
    participant User as U
    participant Frontend as F
    participant Backend as B

```



```
participant Database as D
```

```
U->>F: Navigates to lead submission page
```

```
F->>F: Renders page, triggers call for item data
```

```
F->>B: Sends GET request (/items)
```

```
B->>D: Retrieves items data from database
```

```
D-->>B: Sends items data
```

```
B-->>F: Returns items data (200 OK)
```

```
F-->>U: Displays lead submission form with item options
```

```
U->>F: Enters lead information
```

```
F->>F: Validates user input
```

```
F->>B: Sends POST request (/leads)
```

```
B->>D: Validates customer and item details
```

```
alt Validation Success
```

```
    D-->>B: Send success response
```

```
    B-->>F: Success response (200 OK)
```

```
    F-->>U: Displays success message
```

```
else Validation Failure
```

```
    D-->>B: Send failure response
```

```
    B-->>F: Failure response (400 Bad Request)
```

```
    F-->>U: Displays error message
```

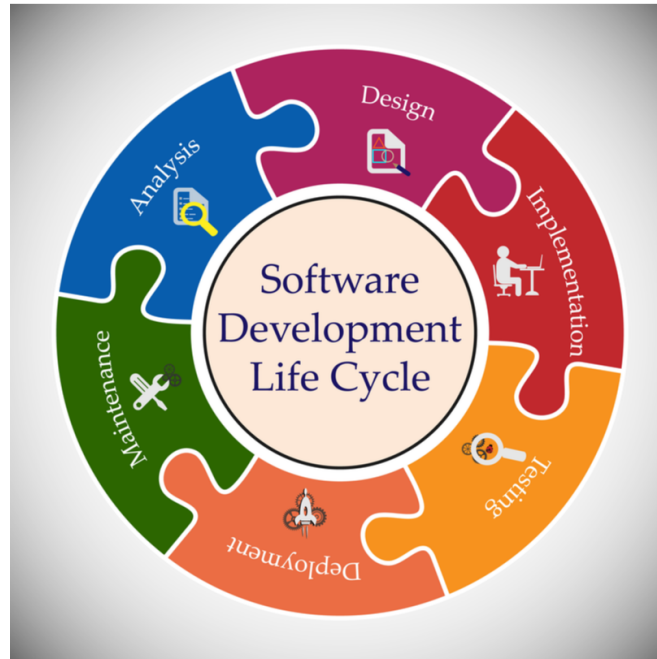
```
end
```

Repositories

SDLC in context

SDLC

The Software Development Life Cycle (SDLC) is a structured approach to software development, encompassing various stages from initial planning to deployment and maintenance.



In the context of this project involving lead generation, SDLC phases align as follows:

1. Planning
 - a. Requirement Gathering: Gather user requirements, functionalities needed for the lead submission platform, and infrastructure needs.
 - b. Scope Definition: Define the scope of the project, including features, technology stack, and deliverables.
2. Design
 - a. System Design: Design the system architecture, including frontend, backend, infrastructure, and database structures.
 - b. Detailed Design: Create detailed design documents, UI wireframes, and technical specifications for development.
3. Implementation (Development)
 - a. Frontend Development: Develop the lead submission web page using React, TypeScript, and Tailwind CSS based on the design specifications.
 - b. Backend Development: Implement backend services using Spring Boot, integrating APIs, and handling lead submissions and database interactions.
 - c. Infrastructure Setup: Use Terraform to set up AWS infrastructure components as defined in the design phase.
4. Testing
 - a. Unit Testing: Conduct unit tests for frontend and backend functionalities to ensure individual components work as expected.
 - b. Integration Testing: Test interactions between frontend, backend, and infrastructure components to ensure they work together seamlessly.
 - c. User Acceptance Testing (UAT): Engage users or stakeholders to perform UAT to validate if the system meets their requirements.
5. Deployment

- a. Deployment Planning: Plan deployment strategies for frontend to S3, backend to ECS, and infrastructure setup in AWS.
 - b. CI/CD Configuration: Set up GitHub Actions for continuous integration and deployment to automate the deployment process.
- 6. Operations and Maintenance
 - a. Monitoring and Optimization: Monitor system performance using CloudWatch, optimise infrastructure, and address performance bottlenecks.
 - b. Bug Fixing: Address any reported issues or bugs in the system post-deployment.
 - c. Enhancements and Updates: Implement additional features or improvements based on user feedback or evolving requirements.
- 7. Iteration
 - a. Continuous Improvement: Iterate the system based on user feedback, technological advancements, or changing business needs to ensure its relevance and efficiency.
 - b. Feedback Integration: Integrate user feedback and lessons learned into future iterations for ongoing improvement.

The SDLC guides the project through distinct phases, ensuring a structured approach to development, testing, deployment, and maintenance. It aligns with the iterative nature of design thinking and the adaptable nature of the project's requirements and user-centric approach.

Relationship with Design Thinking

SDLC and design thinking, while distinct methodologies, can complement each other effectively in software development projects like the lead generation system:

Overlapping Elements

- 1. User-Centric Approach:
 - a. Design Thinking: Emphasises understanding user needs, empathy, and user involvement throughout the process.
 - b. SDLC: Focuses on gathering requirements, ensuring they align with user needs and expectations.
- 2. Iterative Nature:
 - a. Design Thinking: Encourages iteration, prototyping, and continuous improvement based on user feedback.
 - b. SDLC: Often includes iterative cycles (e.g., Agile methodologies) allowing for incremental development and adaptation to changes.
- 3. Problem-Solving and Creativity:
 - a. Design Thinking: Encourages brainstorming, creativity, and innovative problem-solving approaches.
 - b. SDLC: Encourages a structured approach to problem-solving, utilising methodologies and frameworks for development.

Integration Points

1. Requirement Analysis:
 - a. Design Thinking: Emphasises understanding user needs and defining problems from a user perspective.
 - b. SDLC: Involves requirements gathering, analysis, and scoping for development, ensuring alignment with user needs.
2. Prototyping and Testing:
 - a. Design Thinking: Focuses on prototyping, user testing, and iteration based on feedback.
 - b. SDLC: Includes testing phases like unit testing, integration testing, and user acceptance testing (UAT) to ensure functionalities align with requirements.
3. Continuous Improvement:
 - a. Design Thinking: Aims for continuous refinement, adaptation, and innovation based on user feedback and insights.
 - b. SDLC: Encourages a cycle of continuous improvement, enabling enhancements and updates to the system based on feedback and evolving requirements.

Synergistic Approach

1. **Design Thinking as a Foundation:** Design thinking can lay the groundwork for understanding user needs and ideating solutions.
2. **SDLC as a Framework:** SDLC provides a structured approach to development, testing, deployment, and maintenance.

Integration in the Project

1. **Early Stages:** Design thinking methodologies like empathising, defining, and ideating align with the initial phases of requirement gathering and analysis in SDLC.
2. **Middle Stages:** Prototyping, user testing, and iterative improvements in design thinking align with the development, testing, and deployment phases in SDLC.
3. **Late Stages:** Continuous improvement and feedback integration in design thinking align with the maintenance and iteration phases in SDLC.

By integrating design thinking's user-centric approach, creativity, and iteration with SDLC's structured development, testing, and deployment processes, projects can achieve a balance between innovation and systematic development, resulting in user-friendly and robust systems.